

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА - Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий
Кафедра вычислительной техники (ВТ)

Отчёт

по дисциплине

«Разработка программно-аппаратного обеспечения информационных и автоматизированных систем»

Выполнил студент группы ИММО-01-25 Константинов И. С.

Принял старший преподаватель Скрябин Ю. М.

Работа выполнена «15» ноября 2025 г.

«Зачтено» « » _____ 2025 г.

Москва 2025

Введение

Целью данной работы является проектирование и программная реализация эмулятора учебного процессора, а также закрепление навыков низкоуровневого программирования путем написания алгоритмов на языке ассемблера для разработанной архитектуры. Актуальность работы обусловлена необходимостью глубокого понимания принципов функционирования вычислительных систем, механизмов адресации памяти и взаимодействия аппаратных компонентов процессора.

В рамках работы была поставлена задача создать приложение «Мини-Эмулятор» с графическим интерфейсом, поддерживающее загрузку машинного кода, пошаговую отладку и визуализацию состояния регистров и памяти. В качестве демонстрации возможностей платформы были реализованы два алгоритма: вычисление суммы элементов вектора и свертка двух массивов.

Разработанная система базируется на Гарвардской архитектуре, где память команд и память данных разделены. Это позволяет оптимизировать процесс выборки инструкций и работу с данными.

Память данных (ОЗУ): Организована в виде массива mem фиксированного размера в 128 ячеек (адреса 0..127). Каждая ячейка хранит целочисленное значение (Python int), которое может интерпретироваться как знаковое число. Память используется для хранения исходных массивов, промежуточных вычислений и записи финальных результатов.

Память команд: Программа хранится отдельно в виде списка инструкций. Это исключает возможность случайной модификации кода программы как данных в процессе выполнения.

Регистровый файл: Процессор оснащен блоком из 8 регистров общего назначения (R0–R7). Несмотря на универсальность, регистры имеют конвенциональное назначение для удобства разработки ПО:

R0 (Аккумулятор): используется для накопления результатов арифметических операций.

R1 (Tmp): регистр временного хранения.

R2 (Counter): счетчик итераций в циклах.

R3 (Pointer): указатель адреса при работе с массивами.

R4–R7: регистры свободного назначения.

1.2. Управляющие регистры и цикл процессора

Управление потоком выполнения осуществляется с помощью специальных регистров:

PC (Program Counter): счетчик команд, хранящий индекс текущей инструкции в списке программы. После выборки каждой команды PC автоматически инкрементируется (PC += 1), если не происходит перехода.

IR (Instruction Register): регистр команд, содержащий текущую выполняемую инструкцию в разобранном виде — кортеж (opcode, operand_a, operand_b, operand_c).

Логика работы эмулятора построена на классическом цикле выборки-исполнения:

Fetch (Выборка): извлечение инструкции по адресу PC.

Decode (Декодирование): определение типа операции.

Execute (Исполнение): выполнение действия (ALU, доступ к памяти) и обновление флагов/регистров.

Система команд и формат инструкций

Система команд эмулятора спроектирована как 4-х адресная. Формат инструкции включает код операции (Opcode) и три операнда: регистр назначения (Rdst) и два источника (Rsrc1, Rsrc2).

Ниже приведен перечень реализованных инструкций:

LOAD (код 1): Загрузка данных из памяти данных в регистр. Используется прямая или косвенная адресация.

ADD (код 2): Сложение значений двух регистров. Результат помещается в регистр назначения.

INC/DEC (коды 3/4): Инкремент и декремент значения регистра. Необходимы для управления счетчиками циклов и указателями.

MOV (код 5): Копирование данных между регистрами.

JNZ (код 6): Условный переход (Jump if Not Zero). Если проверяемый регистр не равен нулю, в PC записывается новый адрес. Это основной механизм реализации циклов[9].

STORE (код 9): Сохранение значения из регистра в ячейку памяти данных.

HALT (код 255): Завершение работы программы.

Реализация программного интерфейса

Приложение реализовано на языке Python с использованием графической библиотеки. Интерфейс разделен на три функциональные зоны (Рис. 1):

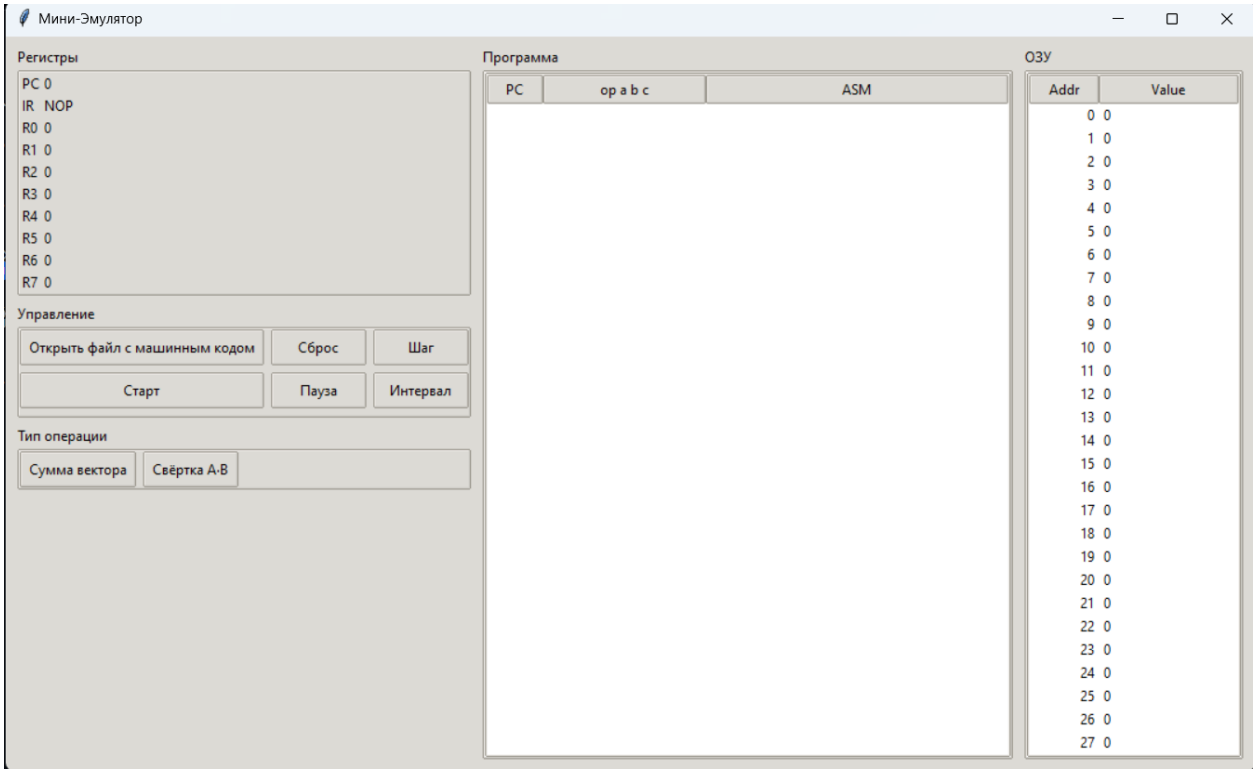


Рисунок 1 – Интерфейс программы

Панель регистров (слева): Отображает текущие значения PC, IR и всех регистров R0-R7. Это позволяет отслеживать изменение состояния процессора на каждом такте.

Панель управления (слева внизу): Содержит элементы управления эмуляцией (Рис. 2):

Открыть файл: загрузка машинного кода.

Старт: автоматическое выполнение программы.

Шаг: выполнение одной инструкции (отладка).

Сброс: возврат процессора в исходное состояние.

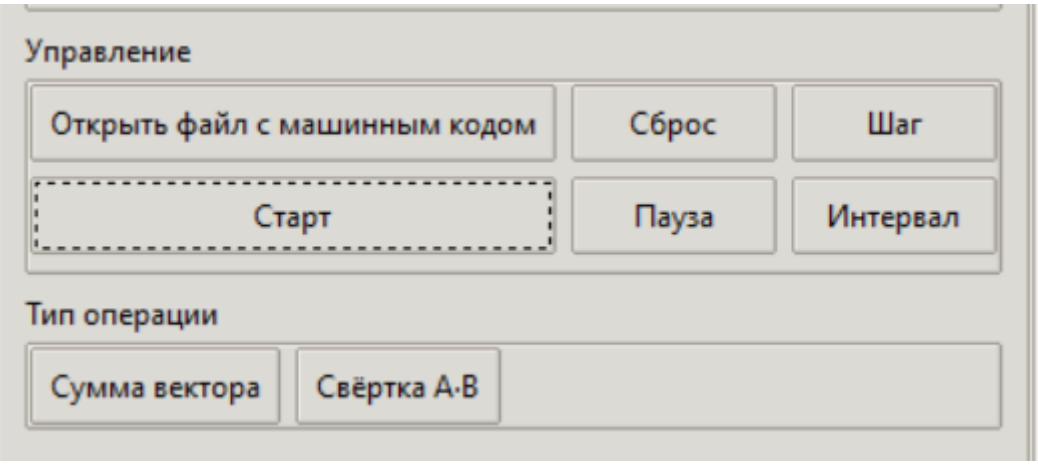


Рисунок 2 – Панель управления приложения

Окно программы (центр): Визуализирует загруженный код. В таблице отображаются адрес, машинный код (op a b c) и мнемоническое представление (Assembler) для удобства чтения (Рис. 3).

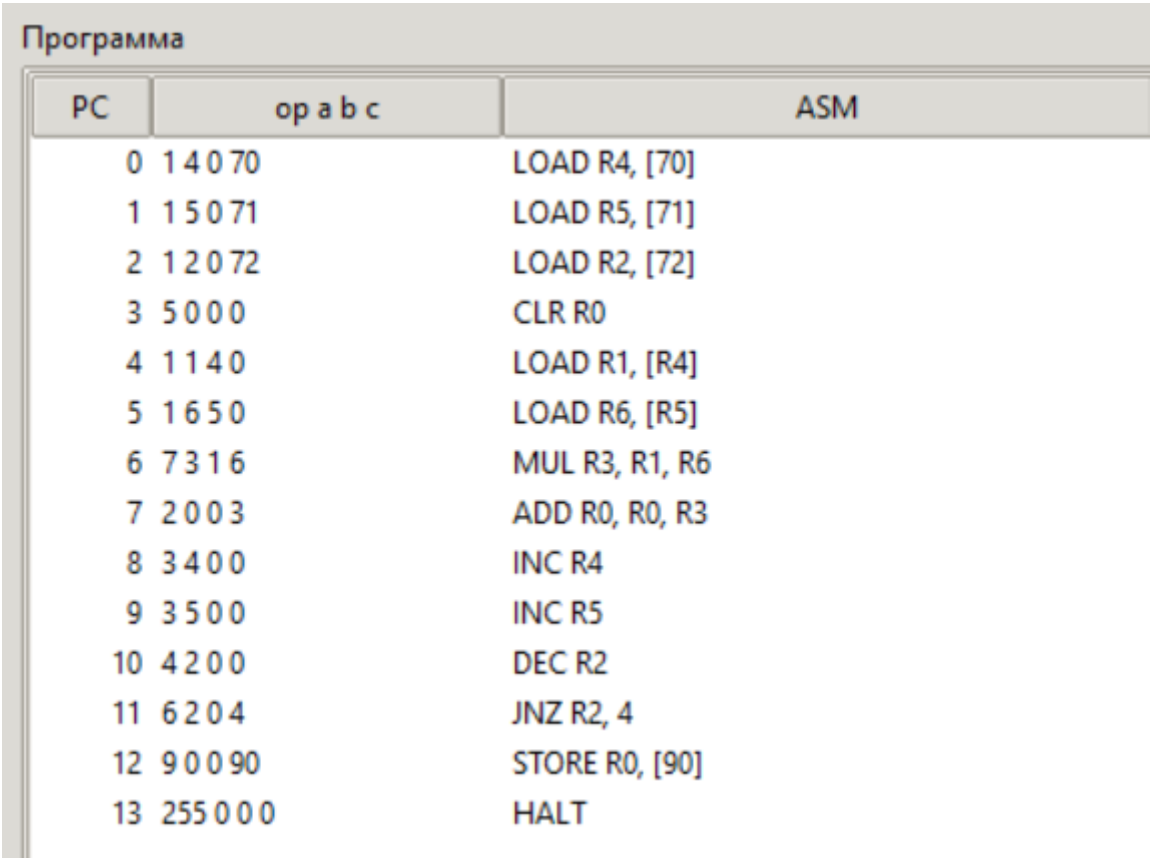


Рисунок 3 – Визуализация загруженного кода

Окно ОЗУ (справа): Таблица, показывающая содержимое всех 128 ячеек памяти. Позволяет наблюдать за входными данными и результатами записи (Рис. 4).

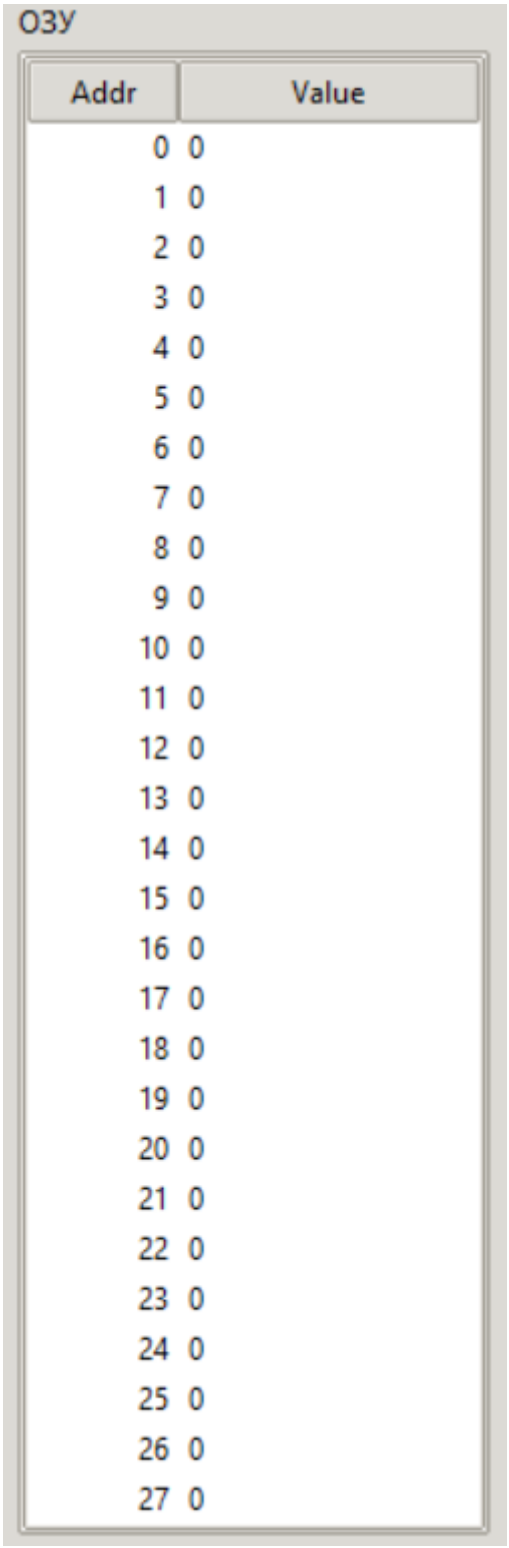


Рисунок 4 – Визуализая ОЗУ

Разработка и тестирование алгоритмов

Для верификации работы эмулятора были разработаны две программы на ассемблере.

Алгоритм «Сумма вектора»

Задача заключается в нахождении суммы элементов целочисленного массива. Алгоритм реализации на ассемблере представлен на рис. 5.

```
%include "io.inc"

section .data
Mass db 6, 12, -6, 23, 14, -9, 10
N db 7
Sum dw 0

section .text
global CMAIN
CMAIN:
mov ebp, esp

xor eax, eax
xor ecx, ecx
xor esi, esi

mov cl, [N]
dec cl
mov esi, Mass
inc esi
xor ax, ax

M1:
movsx bx, byte [esi]
add ax, bx
inc esi
dec cl
jnz M1

mov [Sum], ax
PRINT_DEC 2, ax
NEWLINE

ret
```

Рисунок 5 – Реализация суммы векторов на ассемблере

В памяти данных инициализируется массив и переменная длины вектора. Программа загружает длину вектора в регистр-счетчик R2 и адрес начала массива в R313.

В теле цикла происходит:

Чтение элемента массива по адресу из R3.

Сложение элемента с аккумулятором R0.

Инкремент указателя R3 и декремент счетчика R2.

Проверка условия выхода из цикла (JNZ R2).

После завершения цикла результат сохраняется в память по адресу 39. Тестовый прогон программы показал корректный результат: сумма элементов равна 44 (Рис. 6).

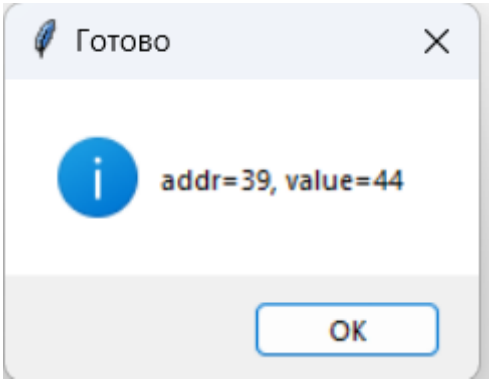


Рисунок 6 – Сообщение о завершении работы программы

Алгоритм «Свертка массивов»

Более сложная вычислительная задача, требующая синхронного прохода по двум массивам. Реализация ее на ассемблере представлена на рис. 7.

convolution.asm

```
%include "io.inc"

section .data
A db 5, -3, 12, -8, 4, 9, -2, 7, -11, 6
B db -4, 10, 3, -6, 2, 5, 8, -9, 7, -1
N db 10
S dw 0

section .text
global CMAIN
CMAIN:
    mov ebp, esp

    xor eax, eax
    xor ebx, ebx
    xor ecx, ecx
    xor esi, esi
    xor edi, edi

    mov cl, [N]
    mov esi, A
    mov edi, B
    xor ax, ax

M1:
    movsx bx, byte [esi]
    movsx dx, byte [edi]
    imul bx, dx
    add ax, bx
    inc esi
    inc edi
    dec cl
    jnz M1

    mov [S], ax
    PRINT_DEC 2, ax
```

Рисунок 7 – Реализация свертки массива на ассемблере

Программа считывает элементы двух векторов (A и B), перемножает их и накапливает сумму произведений. Для реализации умножения, отсутствующего в базовом наборе команд как отдельная инструкция, может использоваться программная эмуляция или расширенная версия АЛУ.

Результат вычислений для тестовых данных составил -75, что было успешно записано в ячейку памяти по адресу 90 (Рис. 8).

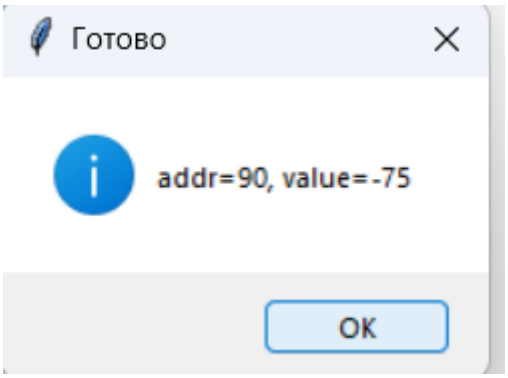


Рисунок 8 – Вывод работы программы

Заключение

В ходе выполнения работы был спроектирован и реализован программный эмулятор процессора Гарвардской архитектуры. Разработанный инструмент обеспечивает полную наблюдаемость вычислительного процесса благодаря визуализации памяти и регистров. Написанные и отлаженные программы (сумма вектора и свертка) подтвердили корректность реализации системы команд и механизмов адресации. Эмулятор может быть использован в качестве учебного пособия для изучения основ организации ЭВМ.